Enterprise-Grade Multi-Location Inventory Control System Design

This document synthesizes the comprehensive design specification for a deterministic inventory control system tailored for multi-location restaurant groups. The system is engineered to function as a control mechanism, shaping behavior over decades rather than merely tracking transactions. It emphasizes event-sourcing, immutability, and a design philosophy focused on long-term sustainability and cost-efficiency within AWS Free Tier limits.

1. Business Context

Target Profile

The system is designed for restaurant groups with an initial size of 1-5 locations, scalable to 20+ locations. Each location typically features 50-100 seats, serves 200-300 covers per day, and actively stocks 40-60 unique ingredients. Staffing per location includes 2-3 managers (for data entry, reconciliation), 5-10 line cooks (for consumption tracking), and 1 location manager. Corporate oversight involves 1 owner/operator, 1 accountant, and an optional auditor [1].

Current State

The system is intended to replace existing methods such as Excel sheets, QuickBooks inventory modules, or manual counts. Historical data from these legacy systems may be incomplete, inconsistent, or paper-based. Users are generally not software engineers. Some locations may already utilize POS systems (e.g., Toast, Square), while others may not [1].

Expected Usage (per location, per month)

- **Events**: Approximately 800 events per day, totaling 24,000 events per month.
- **Sales Uploads**: 30 files per month (one daily upload).
- **Physical Counts**: 30 daily (End-of-Day), 4 weekly (full), and 1 monthly audit.
- **Transfers between locations**: 10-50 per month, depending on the operational model [1].

Multi-Location Scaling (5 locations)

- **Events**: 120,000 events per month in total.
- **DynamoDB Writes**: Approximately 150,000 per month (well within AWS Free Tier limits).

- **Lambda Invocations**: Approximately 500,000 per month (50% of Free Tier limit) [1].

Success Metrics

- **Inventory Variance**: Less than 2% by value per location within 90 days.
- **Count-to-Reconciliation Workflow**: Less than 15 minutes per location.
- **Correction Rate**: Decreases month-over-month, indicating improved behavior.
- **System Trust Score**: Greater than 4/5 per location (based on monthly user surveys).
- **Cross-Location Transfer Accuracy**: Greater than 95% (items sent equal items received) [1].

2. Core Philosophy

The fundamental principle is to **design control systems, not merely applications**. The system aims to shape behavior over decades, ensuring longevity and adaptability.

Assumptions (Iron Laws)

1. **Humans adapt to systems**: Design for desired behavior.
2. **Shortcuts emerge under pressure**: Make shortcuts visible, not easy.
3. **Incentives matter**: Align the system with operational goals.
4. **Infrastructure is scarce**: AWS Free Tier is a hard constraint.
5. **Bugs reach production**: Design for repair, not perfection.
6. **Locations operate semi-independently**: Design for network partition tolerance.
7. **Sales data is the source of truth for consumption**: Recipes are theoretical models.
8. **The system will outlive the current technology stack**: Design for 20+ years [1].

Thinking Framework

First-Order Effects (What the system does)

- Correct inventory accounting per location and consolidated.
- Accurate cost attribution (COGS per location, total).
- Reproducible financial reports.
- Track inter-location transfers with full accountability.
- Deduce consumption from sales data + recipe definitions [1].

Second-Order Effects (How the system shapes behavior)

- Discourage sloppy counting (make errors explicit per location).
- Prevent silent workarounds (make bypasses harder than correct usage).
- Avoid variance fatigue (don't overwhelm with noise).
- Shape disciplined operational habits over time.
- Encourage accurate transfer documentation (accountability prevents shortcuts).
- Surface recipe inaccuracies through variance (when theoretical ≠ actual, investigate recipe) `1`.

Inversion Principle (What you must prevent)

- Loss of trust in numbers (once lost, hard to regain).
- Normalization of corrections (corrections should be rare, not routine).
- Report proliferation without meaning (every report must answer a question).
- Humans bypassing the system (if they bypass it, the system failed).
- "Temporary" overrides becoming permanent (no backdoors).
- Location rivalry undermining consolidated accuracy (blame-shifting on variances).
- Transfer discrepancies being "resolved" by silent adjustments (must be explicit).
- Sales data entry becoming a bottleneck (must be fast, forgiving, correctable) `1`.

Core Invariants (Non-Negotiable)

1. **Inventory is an immutable event ledger per location**: No DELETE, only append.
2. **All state is derived, never mutated**: `balance(location, item) = f(events)`.
3. **All calculations are pure and deterministic**: Same inputs → same outputs, always, forever.
4. **All reports are reproducible by replay**: Can regenerate any historical report exactly.
5. **Human input is noisy sensor data**: Validate, version, timestamp everything.
6. **Corrections are additive and time-bounded**: Corrections are events, decay in relevance.
7. **Failure degrades locally**: One location failure doesn't cascade to others.
8. **Scarcity is a design constraint**: AWS Free Tier limits are features, not bugs.
9. **No hidden defaults, ever**: Explicit > implicit.
10. **Determinism = mathematical equivalence**: Same calculation, same result, provably.

11. **Locations can operate offline**: Eventual consistency across locations acceptable.
12. **Sales data drives consumption**: Theoretical recipes validated against actual usage.
13. **Every transfer has two events**: Sent AND received (never orphaned).
14. **System evolution is forward-compatible**: Old events readable by new code forever [1].

3. Domain Model (Do Not Merge Concerns)

The system is structured around 10 distinct domains, each with specific responsibilities and invariants.

1. **Inventory Ledger Domain**
   - **Owns**: Event log, snapshots, balances per location.
   - **Events**: `PurchaseReceived`, `RecipeExecuted`, `PhysicalCountRecorded`, `TransferSent`, `TransferReceived`, `WasteRecorded`, `CorrectionApplied`, `InitialInventoryEstablished`.
   - **Invariant**: `Balance(location, item, time) = f(events at location up to time)` [1].
2. **Procurement Domain**
   - **Owns**: Purchase orders, vendor invoices, receiving.
   - **Events**: `PurchaseOrderCreated`, `InvoiceReceived`, `PurchaseReceivedAtDock`.
   - **Invariant**: Every item in inventory has a procurement event (or initial inventory event) [1].
3. **Production & Consumption Domain**
   - **Owns**: Recipes, bills of material, yields, production records, sales data.
   - **Events**: `RecipeVersioned`, `ProductionBatchStarted`, `IngredientConsumed`, `YieldRecorded`, `SalesDataUploaded`, `SalesRecorded`, `ConsumptionDeduced`.
   - **Invariant**: Every ingredient consumed ties to sales data → recipe → consumption OR waste event [1].
4. **Counting & Reconciliation Domain**
   - **Owns**: Physical counts, variance detection, classification.
   - **Events**: `CountInitiated`, `ItemCounted`, `VarianceDetected`, `VarianceClassified`, `ReconciliationClosed`.
   - **Invariant**: `Variance = Physical - Theoretical` (with confidence scores) [1].
5. **Financial Attribution Domain**
   - **Owns**: Cost assignment, COGS calculation, valuation, per-location P&L.

- **Events**: `CostAssigned`, `COGSCalculated`, `ValuationSnapshot`, `LocationProfitabilityComputed`.
- **Invariant**: `Total inventory value = Σ(quantity × unit cost)` using FIFO/Weighted method `1`.

6. **Reporting & Determinism Domain**
   - **Owns**: Report definitions, versioning, replay, dashboard configurations.
   - **Events**: `ReportGenerated`, `ReportVersionUpdated`, `DashboardConfigured`.
   - **Invariant**: `Report(input_v1, code_v1, dashboard_config_v1)` always produces the same output `1`.
7. **Governance & Control Domain**
   - **Owns**: Policies, approval workflows, audit trail, role-based dashboards.
   - **Events**: `PolicyEnforced`, `CorrectionApproved`, `AuditTrailRequested`, `RolePermissionChanged`.
   - **Invariant**: All sensitive operations are logged and attributed `1`.
8. **Observability & Repair Domain**
   - **Owns**: Metrics, anomaly detection, repair workflows, determinism validation.
   - **Events**: `AnomalyDetected`, `RepairInitiated`, `EventQuarantined`, `ReplayDivergenceDetected`.
   - **Invariant**: System observability fits within Free Tier limits `1`.
9. **Location & Transfer Domain (NEW)**
   - **Owns**: Location registry, transfer tracking, cross-location reconciliation.
   - **Events**: `LocationRegistered`, `TransferInitiated`, `TransferSent`, `TransferReceived`, `TransferReconciled`, `TransferDiscrepancyDetected`.
   - **Invariant**: For every `TransferSent`, there must eventually be a `TransferReceived` (or `TransferLost`) `1`.
10. **Sales Integration Domain (NEW)**
    - **Owns**: Sales data ingestion, parsing, validation, menu-to-recipe mapping.
    - **Events**: `SalesFileUploaded`, `SalesFileParsed`, `SalesRecorded`, `MenuItemMapped`, `SalesDataRejected`.
    - **Invariant**: Sales data is idempotent (uploading the same file twice results in the same events) `1`.

4. Event Taxonomy (Concrete Examples with Multi-Location & Sales)

Core Event Structure (unchanged)

TypeScript

```
interface Event {
  event_id: string; // UUIDv7 (time-sortable, deterministic from timestamp+sequ
  event_type: string;
  event_time: ISO8601; // Business time (when it happened)
  processing_time: ISO8601; // System time (when recorded)
  correlation_id: string; // Request tracing
  causation_id: string; // Which event caused this
  actor_id: string; // Who/what caused it
  location_id: string; // NEW: Every event has a location (or "corporate")
  version: number; // Schema version
  payload: object; // Event-specific data
}
```

New Event Examples

- **LocationRegistered** 1

  JSON

  ```
  {
    "event_id": "01HWGXYZ...",
    "event_type": "LocationRegistered",
    "event_time": "2025-01-15T00:00:00Z",
    "processing_time": "2025-01-15T09:00:00Z",
    "correlation_id": "onboarding-location-downtown",
    "causation_id": null,
    "actor_id": "user:owner-charlie",
    "location_id": "corporate", // Corporate-level event
    "version": 1,
    "payload": {
      "location_id": "loc:downtown",
      "name": "Downtown Location",
      "address": "123 Main St, City, State 12345",
      "timezone": "America/New York",
      "opened_date": "2020-03-01",
      "status": "active"
    }
  }
  ```

- **SalesFileUploaded** 1

  JSON

```
{
  "event_id": "01HWGY04...",
  "event_type": "SalesFileUploaded",
  "event_time": "2025-04-02T23:00:00Z", // EOD upload
  "processing_time": "2025-04-02T23:00:15Z",
  "correlation_id": "sales-upload-0402",
  "causation_id": null,
  "actor_id": "user:manager-alice",
  "location_id": "loc:downtown",
  "version": 1,
  "payload": {
    "file_id": "sales-2025-04-02.xlsx",
    "file_hash": "sha256:abc123...", // For idempotency
    "upload_timestamp": "2025-04-02T23:00:00Z",
    "status": "pending_parse"
  }
}
```

- **SalesFileParsed** 1

JSON

```
{
  "event_id": "01HWGY05...",
  "event_type": "SalesFileParsed",
  "event_time": "2025-04-02T23:00:00Z",
  "processing_time": "2025-04-02T23:00:20Z",
  "correlation_id": "sales-upload-0402",
  "causation_id": "01HWGY04...",
  "actor_id": "system:sales-parser",
  "location_id": "loc:downtown",
  "version": 1,
  "payload": {
    "file_id": "sales-2025-04-02.xlsx",
    "rows_parsed": 45,
    "rows_valid": 43,
    "rows_invalid": 2,
    "invalid_rows": [
      { "row": 12, "error": "Unknown menu item: 'Pasta Primavera'" },
      { "row": 31, "error": "Invalid quantity: 'five' (expected number)" }
    ],
    "status": "partial_success" // Or "success" if no invalid rows
  }
}
```

- **SalesRecorded** (individual menu item sale) 1

JSON

```
{
  "event_id": "01HWGY06...",
  "event_type": "SalesRecorded",
  "event_time": "2025-04-02T18:30:00Z", // Sale timestamp from Excel
  "processing_time": "2025-04-02T23:00:21Z",
  "correlation_id": "sales-upload-0402",
  "causation_id": "01HWGY05...",
  "actor_id": "system:sales-parser",
  "location_id": "loc:downtown",
  "version": 1,
  "payload": {
    "sale_date": "2025-04-02",
    "menu_item_id": "menu:chicken-parmesan",
    "menu_item_name": "Chicken Parmesan",
    "quantity_sold": 12,
    "unit_price": 18.99, // Optional, for revenue tracking
    "total_revenue": 227.88, // Optional
    "source_file": "sales-2025-04-02.xlsx",
    "source_row": 5
  }
}
```

- **ConsumptionDeduced** (from sales + recipe) 1

JSON

```
{
  "event_id": "01HWGY07...",
  "event_type": "ConsumptionDeduced",
  "event_time": "2025-04-02T18:30:00Z", // Same as sale time
  "processing_time": "2025-04-02T23:00:22Z",
  "correlation_id": "sales-upload-0402",
  "causation_id": "01HWGY06...",
  "actor_id": "system:consumption-engine",
  "location_id": "loc:downtown",
  "version": 1,
  "payload": {
    "sale_event_id": "01HWGY06...",
    "menu_item_id": "menu:chicken-parmesan",
    "recipe_id": "recipe:chicken-parmesan:v3", // Recipe version at sale tim
    "portions_produced": 12,
    "ingredients_consumed": [
      {
        "ingredient_id": "ing:chicken-breast",
        "quantity_theoretical": 6.0, // From recipe
        "quantity_deduced": 6.0, // Exact from recipe (no variance yet)
```

```
        "unit": "lb"
      },
      {
        "ingredient_id": "ing:parmesan",
        "quantity_theoretical": 0.5,
        "quantity_deduced": 0.5,
        "unit": "lb"
      }
    ],
    "deduction_confidence": "high" // Based on recipe accuracy history
  }
}
```

- **TransferInitiated** (from downtown to suburban) 1

JSON

```
{
  "event_id": "01HWGY08...",
  "event_type": "TransferInitiated",
  "event_time": "2025-04-03T10:00:00Z",
  "processing_time": "2025-04-03T10:00:05Z",
  "correlation_id": "transfer-dt-to-sub-001",
  "causation_id": null,
  "actor_id": "user:manager-alice",
  "location_id": "loc:downtown", // Initiating location
  "version": 1,
  "payload": {
    "transfer_id": "xfer:dt-sub-20250403-001",
    "from_location_id": "loc:downtown",
    "to_location_id": "loc:suburban",
    "items": [
      {
        "ingredient_id": "ing:chicken-breast",
        "quantity": 10.0,
        "unit": "lb",
        "unit_cost": 3.45, // Cost at source location
        "total_cost": 34.50
      }
    ],
    "initiated_by": "user:manager-alice",
    "expected_delivery": "2025-04-03T14:00:00Z",
    "status": "pending_send"
  }
}
```

- **TransferSent** (inventory leaves source location) 1

JSON

```
{
  "event_id": "01HWGY09...",
  "event_type": "TransferSent",
  "event_time": "2025-04-03T11:00:00Z", // Physical departure time
  "processing_time": "2025-04-03T11:00:10Z",
  "correlation_id": "transfer-dt-to-sub-001",
  "causation_id": "01HWGY08...",
  "actor_id": "user:manager-alice",
  "location_id": "loc:downtown",
  "version": 1,
  "payload": {
    "transfer_id": "xfer:dt-sub-20250403-001",
    "items_sent": [
      {
        "ingredient_id": "ing:chicken-breast",
        "quantity": 10.0,
        "unit": "lb"
      }
    ],
    "sent_by": "user:manager-alice",
    "sent_at": "2025-04-03T11:00:00Z",
    "carrier": "driver:john", // Optional
    "expected_arrival": "2025-04-03T14:00:00Z"
  }
}
```

- **TransferReceived** (inventory arrives at destination) 1

JSON

```
{
  "event_id": "01HWGY10...",
  "event_type": "TransferReceived",
  "event_time": "2025-04-03T14:15:00Z", // Actual arrival time
  "processing_time": "2025-04-03T14:15:30Z",
  "correlation_id": "transfer-dt-to-sub-001",
  "causation_id": "01HWGY09...",
  "actor_id": "user:manager-bob",
  "location_id": "loc:suburban", // Receiving location
  "version": 1,
  "payload": {
    "transfer_id": "xfer:dt-sub-20250403-001",
    "items_received": [
      {
        "ingredient_id": "ing:chicken-breast",
        "quantity_expected": 10.0,
```

```
        "quantity_actual": 9.8, // Discrepancy!
        "unit": "lb",
        "condition": "good" // Or "damaged", "partial"
      }
    ],
    "received_by": "user:manager-bob",
    "received_at": "2025-04-03T14:15:00Z",
    "discrepancy_noted": true
  }
}
```

- **TransferDiscrepancyDetected** 1

JSON

```
{
  "event_id": "01HWGY11...",
  "event_type": "TransferDiscrepancyDetected",
  "event_time": "2025-04-03T14:15:00Z",
  "processing_time": "2025-04-03T14:15:35Z",
  "correlation_id": "transfer-dt-to-sub-001",
  "causation_id": "01HWGY10...",
  "actor_id": "system:transfer-reconciliation",
  "location_id": "corporate", // Corporate-level reconciliation
  "version": 1,
  "payload": {
    "transfer_id": "xfer:dt-sub-20250403-001",
    "from_location": "loc:downtown",
    "to_location": "loc:suburban",
    "discrepancies": [
      {
        "ingredient_id": "ing:chicken-breast",
        "sent_quantity": 10.0,
        "received_quantity": 9.8,
        "variance_quantity": -0.2,
        "variance_percentage": -2.0,
        "variance_probable_value_usd": -0.69,
        "cause": "in_transit_loss" // Or "measurement_error", "theft"
      }
    ],
    "requires_approval": true, // If variance > threshold
    "status": "pending_review"
  }
}
```

- **PhysicalCountRecorded** (now with location) 1

```JSON
{
  "event_id": "01HWGY02...",
  "event_type": "PhysicalCountRecorded",
  "event_time": "2025-04-02T22:00:00Z",
  "processing_time": "2025-04-02T22:05:30Z",
  "correlation_id": "count-session-eod-0402-downtown",
  "causation_id": null,
  "actor_id": "user:manager-alice",
  "location_id": "loc:downtown", // Count is location-specific
  "version": 1,
  "payload": {
    "storage_area": "walk-in-cooler-1", // Sub-location
    "ingredient_id": "ing:chicken-breast",
    "quantity_counted": 38.5,
    "unit": "lb",
    "confidence": "high",
    "count_method": "scale",
    "notes": null
  }
}
```

- **VarianceDetected** (now with consumption deduction context) 1

```JSON
{
  "event_id": "01HWGY03...",
  "event_type": "VarianceDetected",
  "event_time": "2025-04-02T22:05:31Z",
  "processing_time": "2025-04-02T22:05:32Z",
  "correlation_id": "count-session-eod-0402-downtown",
  "causation_id": "01HWGY02...",
  "actor_id": "system:reconciliation-engine",
  "location_id": "loc:downtown",
  "version": 1,
  "payload": {
    "ingredient_id": "ing:chicken-breast",
    "theoretical_quantity": 40.3, // Opening + purchases - sales consumption
    "physical_quantity": 38.5,
    "variance_quantity": -1.8,
    "variance_percentage": -4.47,
    "variance_value_usd": -6.21,
    "contributing_events": {
      "opening_balance": 50.0,
      "purchases": 0,
      "sales_consumption": 6.0, // From ConsumptionDeduced events
      "waste": 3.7,
```

```
      "transfers_out": 0,
      "transfers_in": 0
    },
    "classification": "acceptable_loss",
    "classification_reason": "within_5pct_threshold_for_perishable"
  }
}
```

- **WasteRecorded** 1

```JSON
{
  "event_type": "WasteRecorded",
  "ingredient_id": "ing:chicken-breast",
  "quantity": 2.5,
  "unit": "lb",
  "waste_category": "spoilage",
  "waste_reason": "Expired (past sell-by date)",
  "recorded_by": "user:manager-alice",
  "cost_impact": 8.63
}
```

5. Multi-Location Architecture

Location Topology

- **Corporate Level**
  - Central event ledger (all locations).
  - Consolidated reporting.
  - Cross-location analytics.
  - Transfer reconciliation.
  - Master data management (ingredients, recipes, vendors) 1.
- **Location Level**
  - Local event generation (purchases, sales, counts).
  - Local snapshots (for offline operation).
  - Local dashboards (real-time).
  - Can operate independently if network partition occurs 1.

Data Sovereignty

- **Single Ledger Model (Recommended)**
  - All events stored in a single DynamoDB table.
  - `location_id` is part of the partition key strategy.
  - Enables corporate-level queries without data synchronization.
  - Simpler consistency model [1].
- **Distributed Ledger Model (If Required for Scale/Latency)**
  - Each location has its own DynamoDB table.
  - Corporate-level aggregation via Lambda queries.
  - Transfer events replicated to both locations.
  - More complex, only if single-table limits are reached [1].

Network Partition Handling

- **During Partition (Location Offline)**
  - Location continues generating events locally (browser localStorage, offline PWA).
  - Events timestamped with `event_time` (when they occurred).
  - Queue for later sync [1].
- **After Partition (Reconnected)**
  - Sync local events to the central ledger.
  - Idempotency keys prevent duplicates.
  - Conflict resolution: `event_time` wins (causality preserved).
  - Transfers initiated during partition flagged for review [1].

Transfer Workflow (Detailed)

1. **Initiation (at source)**
   - Manager creates transfer request (`TransferInitiated`).
   - System validates: sufficient inventory at source.
   - Status: "pending_send" [1].
2. **Send (physical departure)**
   - Manager confirms physical departure (`TransferSent`).
   - Inventory deducted from source location immediately.
   - Status: "in_transit" [1].
3. **Receive (physical arrival)**

- Destination manager counts items (`TransferReceived`).
- Inventory added to destination location.
- If discrepancy detected → `TransferDiscrepancyDetected` `1` .

4. **Reconciliation**
   - If variance < 2%: auto-approved, mark as "in_transit_loss".
   - If variance >= 2%: requires owner approval.
   - Owner investigates, approves correction or flags for investigation.
   - Status: "reconciled" or "under_investigation" `1` .

Determinism in Transfers

- Transfer has a unique ID (`xfer:...`) generated at initiation time.
- All events reference `transfer_id` (causality chain).
- Transfer cannot be deleted, only marked as "cancelled" (additive) `1` .

6. Sales Data Integration (Detailed)

Excel Template Specification

- **Required Columns**
  - `sale_date` (YYYY-MM-DD): Date of sale.
  - `menu_item` (text): Exact menu item name (must match system).
  - `quantity_sold` (number): Whole or decimal (e.g., 12, 0.5 for half portion) `1` .
- **Optional Columns**
  - `unit_price` (currency): For revenue tracking.
  - `notes` (text): Free-form notes (e.g., "comp", "employee meal") `1` .
- **Example**

sale_date	menu_item	quantity_sold	unit_price	notes
2025-04-02	Chicken Parmesan	12	18.99	
2025-04-02	Caesar Salad	8	9.99	
2025-04-02	Margherita Pizza	5	14.99	1 comp

Upload Workflow

1. **Step 1: Upload**
   - Manager uploads Excel file via UI.
   - System computes file hash (SHA-256).
   - Creates `SalesFileUploaded` event [1].
2. **Step 2: Parse**
   - Lambda parses Excel (using SheetJS library).
   - Validates each row: `sale_date` is valid, `menu_item` exists in menu catalog, `quantity_sold` is a positive number.
   - Creates `SalesFileParsed` event with validation results [1].
3. **Step 3: Review (if errors)**
   - UI shows invalid rows with error messages.
   - Manager corrects and re-uploads.
   - Idempotency: same file hash = skip processing [1].
4. **Step 4: Record Sales**
   - For each valid row, create `SalesRecorded` event.
   - Events have the same `event_time` as `sale_date` from Excel [1].
5. **Step 5: Deduce Consumption**
   - For each `SalesRecorded` event:
     - Look up `menu_item` → `recipe_id` mapping (versioned).
     - Retrieve recipe definition (e.g., `recipe:chicken-parmesan:v3`).
     - Calculate ingredients consumed (`quantity_sold` × recipe portions).
     - Create `ConsumptionDeduced` event [1].
6. **Step 6: Reconciliation**
   - Compare theoretical inventory (after consumption) with physical count.
   - Generate `VarianceDetected` events [1].

Menu-to-Recipe Mapping

- **Menu Catalog (Master Data)**

```JSON
```

```
{
  "menu_item_id": "menu:chicken-parmesan",
  "name": "Chicken Parmesan",
  "recipe_id": "recipe:chicken-parmesan:v3",
  "effective_from": "2025-03-01",
  "effective_to": null, // Currently active
  "category": "entree",
  "active": true
}
```

- **Versioned Mapping**
  - Menu items map to recipe versions.
  - Mapping can change over time (`MenuItemRemapped` event).
  - Consumption deduction uses the recipe version active at sale time `1`.
- **Edge Cases**
  - Menu item sold but recipe not found: flag as "unmapped_sale", requires manual mapping.
  - Recipe changed mid-day: use recipe version at sale timestamp.
  - Combo items (burger + fries): composite recipe, sum of sub-recipes `1`.

Idempotency (Critical)

- **Problem**: Manager uploads the same sales file twice.
- **Solution**:
  - Compute file hash on upload.
  - Check: `SalesFileUploaded` event with the same hash exists?
  - If yes: return cached parse results, do not create new events.
  - If no: proceed with parse `1`.
- **Determinism**
  - Same Excel file → same file hash → same events.
  - Parsing logic versioned (`parser:v1` , `parser:v2`).
  - Parse results reproducible (same file + same parser version = same output) `1`.

7. Greenfield vs. Brownfield Startup

Greenfield (New Restaurant, No Prior Data)

Phase 1: Master Data Setup (Week 1)

- **Day 1-2: System Configuration**
  1. Create location (`LocationRegistered` event).
  2. Configure timezone, address, operational hours.
  3. Set up user roles (owner, managers, cooks).
  4. Provision AWS resources (DynamoDB table, Lambda functions, S3 bucket) `1` .
- **Day 3-4: Ingredient Catalog**

  5. Import ingredient list (via bulk upload Excel or manual entry).

  6. For each ingredient, create `IngredientRegistered` event:

  `json { "ingredient_id": "ing:chicken-breast", "name": "Chicken Breast (Boneless, Skinless)", "category": "protein", "unit": "lb", "storage_type": "refrigerated", "shelf_life_days": 7, "par_level": 50.0, // Minimum stock level "reorder_point": 30.0 }`
- **Day 5: Vendor Catalog**

  7. Import vendor list.

  8. Create `VendorRegistered` events:

  `json { "vendor_id": "vendor:sysco-123", "name": "Sysco Foods", "contact": "800-555-0100", "payment_terms": "Net 30", "lead_time_days": 2 }`
- **Day 6-7: Recipe Library**

  9. Create recipes (manual entry or bulk import).

  10. Create `RecipeVersioned` events:

  `json { "recipe_id": "recipe:chicken-parmesan:v1", "name": "Chicken Parmesan", "portions": 1, "ingredients": [{ "ingredient_id": "ing:chicken-breast", "quantity": 0.5, "unit": "lb" }, { "ingredient_id": "ing:parmesan", "quantity": 0.04167, "unit": "lb" } // 0.5oz in lb], "prep_time_minutes": 10, "cook_time_minutes": 25, "yield_notes": "Theoretical; adjust based on actual usage" }`

Phase 2: Initial Inventory Count (Week 2)

- **Day 8: Physical Count**

  11. Perform full physical inventory count.

  12. Count every ingredient in every storage area.

  13. Use mobile app (count entry UI) `1` .
- **Day 9: Data Entry**

  14. Enter counts into the system.

  15. Create `InitialInventoryEstablished` events:

```
json { "event_type": "InitialInventoryEstablished", "location_id": "loc:downtown", "ingredient_id": "ing:chicken-breast", "quantity": 50.0, "unit": "lb", "unit_cost": 3.45, // From most recent invoice, or estimated "total_cost": 172.50, "valuation_method": "weighted_average", // Or FIFO "count_date": "2025-01-15", "counted_by": "user:manager-alice", "notes": "Opening inventory for new system" }
```

- **Day 10: Validation**

  16. Review entered data (spot-check counts).

  17. Validate total inventory value (compare to expected investment).

  18. Create `SystemInitializationCompleted` event (audit trail) [1].

Phase 3: Go Live (Week 3)

- **Day 11-14: Training**

  19. Train managers on daily workflows (count, sales upload, reconciliation).

  20. Train cooks on consumption tracking (if manual entry used).

  21. Test scenarios (purchase entry, sales upload, variance handling) [1].

- **Day 15: First Live Day**

  22. Begin normal operations:

  * Receive purchase → `PurchaseReceived` event.

  * Upload sales data → `SalesRecorded` → `ConsumptionDeduced`.

  * Evening count → `PhysicalCountRecorded`.

  * Reconciliation → `VarianceDetected` [1].

- **Day 15-21: Monitor**

  23. Daily review of variance (expect high variance initially).

  24. Tune recipe quantities based on actual usage.

  25. Adjust variance thresholds (classification rules) [1].

Success Criteria (30 days)

- Users comfortable with daily workflow.
- Variance trending down (learning curve).
- No data loss or system downtime.
- Managers trust the numbers [1].

Brownfield (Existing Restaurant, Migrating from Legacy System)

Phase 1: Data Archaeology (Week 1-2)

- **Assessment**
  1. Identify existing data sources: Excel sheets (purchases, sales, counts), QuickBooks inventory module (valuations, COGS), paper invoices, POS system export `1`.
- **Data Extraction**

  2. Export all available data to Excel/CSV format.

  3. Identify data quality issues: Missing dates, inconsistent units, incomplete ingredient lists, unknown vendors `1`.
- **Data Cleaning**

  4. Normalize units (convert to standard: lb, oz, gal, each).

  5. Map legacy ingredient names to new catalog.

  6. Fill missing data with estimates (mark as "estimated") `1`.

Phase 2: Historical Event Reconstruction (Week 3-4)

- **Cutoff Date Selection**

  7. Choose migration cutoff date (e.g., end of previous month). This becomes the "genesis" of the new system `1`.
- **Opening Balance Determination**

  8. Perform full physical count on cutoff date. This is the authoritative opening balance `1`.
- **Historical Event Import (Optional, if valuable)**

  9. Import last 3-6 months of purchase data:

  * Convert to `PurchaseReceived` events.

  * Use invoice date as `event_time`.

  * Mark as "migrated_data" (`actor_id: "system:migration"`) `1`.

  10. Import sales data (if available):

  * Convert to `SalesRecorded` events.

  * Use sale date as `event_time` `1`.
- **Reconciliation of Imported Data**

  11. Compute theoretical inventory from imported events.

  12. Compare with physical count on cutoff date.

  13. Create `MigrationVarianceDetected` event:

  `json { "event_type": "MigrationVarianceDetected", "ingredient_id": "ing:chicken-breast", "theoretical_from_import": 45.0, // Computed from imported events "physical_on_cutoff": 50.0, //`

`Actual count "variance_quantity": 5.0, "variance_percentage": 10.0, "variance_value_usd": 17.25, "reason": "initial_migration_discrepancy", "action_required": "investigate_or_adjust_opening_balance" }`

8. Determinism Proof

This section details how the system ensures determinism across various potential sources of non-determinism.

1. Floating Point Math

- **Problem**: Standard floating-point numbers (e.g., `0.1 + 0.2 != 0.3`) lead to non-deterministic results across different systems or languages.
- **Solution**:
  - Use fixed-point decimals for all financial and inventory calculations (e.g., `Decimal` type in Python, `BigDecimal` in Java).
  - Store quantities as integers representing the smallest unit (e.g., cents for currency, grams for weight) `1`.
- **Determinism Test**: `0.1 + 0.2` always equals `0.3` `1`.

2. Event Ordering

- **Problem**: Events arriving out of order can lead to different final states.
- **Solution**:
  - Events are ordered by `event_time` first, then `event_id` (UUIDv7) for a total, deterministic order.
  - Processing order is strictly enforced based on this canonical order `1`.
- **Determinism Test**: Replaying events in canonical order always produces the same final state `1`.

3. JSON Serialization

- **Problem**: Different JSON serializers may order keys differently, leading to different hashes for the same logical event.
- **Solution**:
  - Canonical JSON: Always sort keys alphabetically when serializing for hashing.
  - Use deterministic JSON libraries (e.g., `json-stable-stringify` npm package) `1`.
- **Determinism Test**: Hash of event remains the same across serializations `1`.

4. UUIDs & Randomness

- **Problem**: UUIDv4 is random; same inputs produce different UUIDs.
- **Solution**:
  - Use UUIDv7: Time-based, sortable, deterministic from timestamp + sequence counter.
  - Idempotency keys: Client provides a deterministic key (e.g., `sales-2025-04-02-row5`).
  - Never use random in calculations: Only for non-deterministic IDs (user session tokens) [1].
- **Determinism Test**: Same input data → same `event_id` (if replayed from scratch) [1].

5. External Dependencies

- **Problem**: API calls, database queries return different data over time.
- **Solution**:
  - All external data becomes events: Capture in immutable event payload.
  - No live queries in calculations: Use snapshots or event log only.
  - Version external data: Vendor price changes → `PriceChanged` event [1].
- **Determinism Test**: Replaying events does not require external systems to be online [1].

6. Code Versioning

- **Problem**: Same input, different code version → different output.
- **Solution**:
  - Version all calculation code: Embed Git SHA in every report.
  - Version business rules: Variance classification v1, v2, v3.
  - Backward-compatible changes only: Old events + new code = same result.
  - Store calculation version in event: Which code version produced this? [1]
- **Determinism Test**: Replay with same code version → same report output [1].

7. Concurrency & Race Conditions

- **Problem**: Two events at the same time, different ordering → different state.
- **Solution**:
  - Single-writer per aggregate: One Lambda writes to one location's inventory at a time.
  - Causality tracking: event B caused by event A (`causation_id`).

    - Event ordering: Use `event_time` + `event_id` for total order.
    - Optimistic concurrency: DynamoDB conditional writes (version check) [1].
- **Determinism Test**: Events processed in any order → same final state (eventually) [1].

8. Timezone Handling

- **Problem**: Same timestamp, different timezone → different date.
- **Solution**:
    - Store all timestamps in UTC (ISO 8601 format).
    - Convert to local timezone only for display.
    - Date boundaries: Use location timezone for "EOD count" (e.g., 22:00 ET = 02:00 UTC next day) [1].
- **Determinism Test**: Same UTC timestamp + same timezone → same local date [1].

9. Schema Evolution

- **Problem**: Old events missing new fields; default values may vary.
- **Solution**:
    - Explicit defaults in code: Never use language defaults (undefined, null).
    - Version schemas: `event.version = 1, 2, 3...`.
    - Migration logic versioned: If version 1, apply default X [1].
- **Determinism Test**: Old events + new code with explicit defaults → same result every replay [1].

10. Rounding & Precision

- **Problem**: Different rounding strategies → different totals.
- **Solution**:
    - Document rounding strategy: "Always round half-up to 2 decimals."
    - Version rounding rules: `rounding:v1`, `rounding:v2`.
    - Test precision: Unit tests for edge cases (0.005, 0.015, etc.) [1].
- **Determinism Test**: Same rounding version → same rounded result [1].

Determinism Validation (Testing Strategy)

- **Automated Tests**

    - **Test 1: Snapshot Replay**: Take snapshot at time T, replay events from genesis to T, compare states. Assert: balances match exactly (within 0.01 tolerance) [1].
    - **Test 2: Idempotency**: Process event E once, then again. Assert: second processing is no-op [1].
    - **Test 3: Commutativity (Where Applicable)**: Process events [A, B, C] then [C, B, A]. Assert: final state is same (if events are independent) [1].
    - **Test 4: Calculation Reproducibility**: Generate report R at time T with code version V, delete, regenerate R' with same inputs and V. Assert: R === R' [1].
    - **Test 5: Cross-Platform Consistency**: Run calculation on AWS Lambda, locally, and in browser. Assert: all three produce the same result [1].
- **Continuous Monitoring**
    - **Daily Replay Check**: Every day at 2 AM, replay yesterday's events, compare with existing snapshot. Alert if divergence > 0.1% [1].
    - **Audit Trail Integrity**: Weekly: compute hash of all events, compare with previous week's hash + new events. Ensures no events were deleted or modified [1].

Determinism Failure Handling

If Replay Divergence Detected:

1. **Isolate the issue**: Identify the first divergent event, compare expected vs. actual state [1].
2. **Root cause analysis**: Code bug? Data corruption? Schema migration issue? [1]
3. **Remediation**: Fix code (new calculation version), quarantine affected events (manual repair), update migration logic [1].
4. **Re-replay**: From last known-good snapshot, validate results, create `RepairCompleted` event [1].
5. **Prevent recurrence**: Add regression test, update determinism checks [1].

9. Role-Based Customizable Dashboards

Dashboard Architecture

- **Dashboard Configuration (Data-Driven)**: Each role has a dashboard config stored as data (not hardcoded). Example:

```JSON
```

```
{
  "dashboard_id": "dash:manager-variance-focus",
  "role": "manager",
  "location_scope": "own_location", // Or "all_locations" for corporate
  "widgets": [
    {
      "widget_id": "widget:variance-trend",
      "widget_type": "line_chart",
      "title": "30-Day Variance Trend",
      "position": { "row": 1, "col": 1, "width": 2, "height": 1 },
      "data_source": "query:variance-daily-by-location",
      "filters": { "location_id": "${user.location_id}", // Dynamic filter
                   "date_range": "last_30_days" },
      "permissions": ["view_variance"]
    },
    {
      "widget_id": "widget:items-needing-count",
      "widget_type": "table",
      "title": "Items Needing Count",
      "position": { "row": 2, "col": 1, "width": 1, "height": 1 },
      "data_source": "query:items-last-count-gt-24h",
      "filters": { "location_id": "${user.location_id}" },
      "actions": [ { "label": "Start Count", "action": "navigate:/count/${in
      "permissions": ["view_counts", "initiate_count"]
    }
  ],
  "version": 1,
  "created_at": "2025-01-15T00:00:00Z",
  "created_by": "system:default-dashboards"
}
```

- **Widget Library (Reusable Components)**: Line Chart, Bar Chart, Table, KPI Card, Alert List, Heatmap [1].

Role-Specific Dashboards

- **Line Cook Dashboard**
  - **Widgets**: Current Inventory Levels (table, color-coded, action: "Request reorder"), Today's Sales (bar chart, updated hourly), Prep Tasks (checklist, mark as complete) [1].
  - **Permissions**: View inventory (read-only), view sales. Cannot see costs, variance, or corrections [1].
- **Manager Dashboard**

- **Widgets**: Variance Trend (line chart, daily, location-specific, drill-down), Items Needing Count (table, last count > 24h, sorted by value, action: "Start count now"), Correction History (table, recent, who/when/why/amount, action: "View details"), Cost Trends (line chart, highlight spikes), Pending Approvals (alert list, high-variance, transfer discrepancies, action: "Approve" or "Investigate") [1].
- **Permissions**: View and edit inventory, view variance and corrections, approve corrections (< $100 value), initiate counts, upload sales data [1].

- **Owner Dashboard**
  - **Widgets**: Consolidated Inventory Value (KPI card, total across all locations, MoM change), Variance by Location (bar chart, compare locations, drill-down), Top High-Value Variances (table, > $50, cross-location analysis, action: "Investigate"), Supplier Performance (table, on-time delivery, invoice accuracy, action: "View supplier details"), P&L Summary (table, revenue, COGS, gross profit per location, variance impact), Transfer Activity (heatmap, frequency, volume, identify bottlenecks) [1].
  - **Permissions**: View all data across all locations, approve high-value corrections (> $100), configure dashboards for all roles, access audit trail, initiate repairs (admin tools) [1].
- **Auditor Dashboard**
  - **Widgets**: Full Event Log (table, paginated, searchable, filters, actions: "View event details", "Download CSV"), Replay Results (table, recent replay checks, status, action: "View divergence details"), Correction Audit Trail (table, all corrections, who/authorized/reason/evidence, action: "Flag for review"), Determinism Checks (KPI cards, last replay status, events checksum, schema version) [1].
  - **Permissions**: Read-only access to all data, cannot edit or approve anything, can export data, can initiate replays (read-only operation) [1].

Dashboard Customization Workflow

- **User Customization (Limited)**: Users can rearrange widgets (drag-and-drop), hide/show widgets, but cannot add new widgets. Customization saved per user (`user_dashboard_config`) [1].
- **Admin Customization (Full)**: Owner can create new dashboards (role-specific or user-specific), add/remove widgets, configure data sources, filters, permissions. Changes are versioned (`DashboardConfigUpdated` event) [1].
- **Example: Manager Creates Custom Dashboard**
  1. Manager navigates to "Dashboard Settings".
  2. Clicks "Create New Dashboard".

3. Selects widgets (e.g., "Variance Trend", "Top 10 High-Value Items").
4. Arranges widgets in grid layout.
5. Saves as "Manager - High-Value Focus".
6. Dashboard config stored with `created_by: "user:manager-alice"` [1].

Dashboard Determinism

- **Problem**: Dashboard shows different data when refreshed.
- **Solution**:
  - Data freshness indicator: "Data as of YYYY-MM-DD HH:MM AM".
  - Point-in-time queries: Dashboard queries specify `as_of_time`.
  - Snapshot-based widgets: Use yesterday's snapshot (deterministic).
  - Real-time widgets: Clearly labeled "Live Data" (non-deterministic by design) [1].
- **Example**:
  - "Variance Trend" widget: Uses daily snapshots (deterministic).
  - "Current Inventory" widget: Uses latest events (live, non-deterministic) [1].

10. System Evolution: 2 Weeks, 2 Years, 20 Years

2 Weeks In (Learning Phase)

- **User Behavior**: High error rate, frequent questions, over-correcting, distrust of numbers [1].
- **System State**: ~10,000 events total, high variance (10-15%), low recipe accuracy, excellent performance [1].
- **Technical Debt**: Feature requests accumulating, edge cases discovered, bug backlog growing [1].
- **Interventions**: Daily check-ins, recipe tuning sessions, adjust variance thresholds, in-app help improvements [1].
- **Success Indicators**: Users completing daily workflows without help, variance trending down, managers approve <5 corrections/day, no data loss or corruption [1].

2 Years In (Mature Phase)

- **User Behavior**: Error rate normalized (~2% variance), workflows habituated, proactive behavior, trust established [1].

- **System State**: ~600,000 events total, variance stable (2-3%), high recipe accuracy, performance degrading slightly (snapshot queries essential) [1].
- **Technical Debt**: Legacy code accumulating, feature creep, database size growing (approaching 10GB in DynamoDB), monitoring overhead [1].
- **Interventions**: Archival strategy (move events >1 year to S3 cold storage), code refactoring, performance optimization (cache queries), user re-training [1].
- **Success Indicators**: Variance <2% consistently, users self-service 95% of tasks, financial reports match general ledger exactly, system trusted by accountant/auditor [1].
- **Expansion (Likely by Year 2)**: Multi-location (2-5 added), POS integration, OCR for invoices, advanced analytics [1].

20 Years In (Legacy Phase)

- **User Behavior**: Original users retired/moved on, new users don't know "the old way", workflows optimized, system is infrastructure [1].
- **System State**: ~6,000,000 events total, variance very stable (<1%), large recipe library, database size 50GB+ (approaching DynamoDB single-table limits) [1].
- **Technical Debt**: Node.js 20 is ancient, Lambda runtime deprecated, DynamoDB pricing changed, dependencies unmaintained [1].
- **Interventions**: Migration to new runtime (rewrite in current Node.js/Rust/Go, replay all events, blue/green deploy), database migration (time-series DB or S3 archival), code archaeology (document "why", preserve logic, comprehensive test coverage) [1].
- **Success Indicators**: System still running (uptime: decades), financial accuracy maintained, migration completed without data loss, users unaffected by underlying tech changes [1].
- **Replacement (Considered but Rejected)**: The event ledger is irreplaceable. Upgrade underlying tech, preserve event model. Preserve: event schema, calculation logic, domain model. Replace: runtime, libraries, UI framework (not business logic) [1].

Design for 20-Year Survival

- **Principles**:
  1. **Event Schema Stability**: Additive changes only, version every schema change, old events readable forever.
  2. **Calculation Logic Documentation**: Inline comments explaining "why", Architectural Decision Records (ADRs), test cases as living documentation.

3. **Minimal External Dependencies**: Avoid libraries with frequent breaking changes, vendor-lock-in acceptable for infrastructure (AWS), core calculations use stdlib only.
4. **Self-Documenting System**: Event log is documentation, reports explain themselves, audit trail is complete.
5. **Future-Proof Archival**: Events stored in JSON, S3 archival for old data, export capability [1].

- **Migration Checklist (Every 5-10 Years)**:
  - Upgrade runtime (Node.js version).
  - Update dependencies (security patches).
  - Refactor deprecated code (no breaking changes).
  - Replay all events (verify determinism preserved).
  - Update documentation (ADRs, runbooks).
  - Re-train users (if UX changed significantly).
  - Benchmark performance (ensure no degradation).
  - Audit financial accuracy (compare with external source) [1].

11. Final Additions & Missing Considerations

1. Batch Operations

- **Use Case**: Initial setup requires importing 100+ ingredients, 50+ recipes.
- **Solution**: Bulk import UI (Excel template), batch event creation, validation, atomic commit (all events succeed or fail) [1].
- **Example Workflow**: Download template, fill data, upload file, system validates, creates batch events, creates `BulkImportCompleted` event [1].

2. Recipe Testing & Development

- **Use Case**: Chef wants to test a new recipe without affecting production inventory.
- **Solution**: Test Mode Recipe (`test_mode: true`), consumption tracked but flagged as "test", excluded from variance calculations, visible in separate "Test Kitchen" dashboard [1].
- **Example Event**: `TestRecipeExecuted` with `test_mode: true` and notes [1].
- **Transition to Production**: Mark recipe as production-ready, create `RecipePromotedToProduction` event. Previous test events remain in log [1].

3. Seasonal Ingredients

- **Use Case**: Some ingredients only available certain months (strawberries, pumpkins).
- **Solution**: Ingredient seasonality metadata (`available_months` , `peak_months` , `off_season_available`). Dashboard warnings (e.g., "Strawberries out of season - consider menu change") [1].

4. Vendor Management (Enhanced)

- **Procurement Workflow**: Low Inventory Alert → Suggested Purchase Order → Create PO (`PurchaseOrderCreated`) → Send to Vendor → Receive Invoice (`InvoiceReceived`) → Receive Items (`PurchaseReceived`) → Reconcile (PO vs Invoice vs Received) [1].
- **Vendor Performance Tracking**: On-time delivery rate, invoice accuracy, quality issues [1].
- **Events**: `PurchaseOrderCreated` , `InvoiceReceived` , `PurchaseReceived` , `InvoiceDiscrepancyDetected` , `DeliveryDelayed` [1].

5. Waste Categorization (Enhanced)

- **Waste Types**: Spoilage, Prep Waste, Customer Mistake, Employee Meal, Theft [1].
- **Variance Thresholds by Waste Type**: Spoilage (< 3%), Prep waste (< 5%), Customer mistake (< 1%), Theft (0%) [1].
- **Event**: `WasteRecorded` with `waste_category` and `waste_reason` [1].

6. Expiration Tracking (FIFO)

- **Use Case**: Ensure oldest items used first (food safety + cost).
- **Solution**: Lot Tracking (lot number, expiry date per purchase), FIFO Queue (consume from oldest lot first), Expiration Alerts (dashboard shows items expiring within 3 days) [1].
- **Events**: `PurchaseReceived` (with lot_number, expiry_date), `LotConsumed` , `ExpirationWarning` [1].
- **Dashboard Widget**: "Items Expiring Soon" (sorted by expiry date), action: "Use in prep today" or "Mark for waste" [1].

7. Compliance & Auditing

- **Health Department Requirements**: Temperature logs, expiration tracking, waste disposal records [1].

- **Financial Audits**: Year-end inventory valuation, COGS reconciliation, variance explanation [1].
- **Tax Reporting**: Inventory value for balance sheet, COGS for P&L, depreciation of inventory [1].
- **Events**: `TemperatureRecorded`, `ComplianceCheckPassed`, `YearEndInventoryValuation` [1].

8. User Training & Documentation

- **In-App Help**: Context-sensitive tooltips, video tutorials, FAQ section [1].
- **Training Mode**: Sandbox environment, users practice without affecting production, reset button [1].
- **Onboarding Checklist**: Intro video, daily count workflow training, sandbox practice, variance review with manager [1].

9. Notification System

- **Alerts (Push Notifications, Email, Slack)**:
  - **High Variance**: "Chicken breast variance 8% - requires investigation". Sent to: Manager, Owner. Frequency: Real-time [1].
  - **Overdue Count**: "5 high-value items not counted in 48 hours". Sent to: Manager. Frequency: Daily (morning) [1].
  - **Low Inventory**: "Chicken breast below reorder point (20 lb remaining)". Sent to: Manager. Frequency: Real-time [1].
  - **Transfer Discrepancy**: "Transfer from Downtown to Suburban: 2% variance - needs approval". Sent to: Manager (both locations), Owner. Frequency: Real-time [1].
- **Configuration**: User preferences (Email vs Slack vs SMS), threshold settings, quiet hours [1].

10. Backup & Disaster Recovery

- **DynamoDB**: Point-in-time recovery (PITR) enabled (35 days), on-demand backups (weekly, 90 days retention), cross-region replication (optional) [1].
- **S3**: Versioning enabled, cross-region replication for reports, lifecycle policies (archive old reports to Glacier) [1].
- **Recovery Scenarios**:
  - **Accidental Data Deletion**: Use PITR to restore DynamoDB, compare, re-add deleted events [1].

    - **AWS Regional Outage**: Failover to secondary region (if replicated), otherwise wait for AWS recovery (events queued locally) [1].
- **Disaster Recovery Test (Quarterly)**: Simulate failure, restore from backup, verify data intact, document recovery time (RTO) [1].

12. Output Deliverables (Comprehensive)

The design specifies the production of the following documents:

1. **Executive Summary (2-3 pages)**: System vision, philosophy, multi-location capabilities, deterministic guarantees, 20-year sustainability, business value proposition [1].
2. **Domain Architecture Diagram**: 10 domains with boundaries, events, dependencies; service decomposition; data flow; multi-location topology [1].
3. **DynamoDB Schema Design**: Single-table layout (PK, SK, GSIs), sample data, access patterns (17 common queries), capacity planning (Free Tier forecast) [1].
4. **Event Catalog (Complete)**: All 30+ event types with JSON schema, event relationships, event versioning strategy, idempotency implementation [1].
5. **Calculation Flow Diagrams**: Theoretical inventory algorithm (pseudocode + flowchart), variance classification logic (decision tree), COGS calculation (FIFO/weighted average), transfer reconciliation [1].
6. **Sales Integration Specification**: Excel template design, upload workflow (5-step), menu-to-recipe mapping, consumption deduction algorithm, error handling [1].
7. **Multi-Location Design**: Location registry, transfer workflow, network partition handling, consolidated reporting [1].
8. **Determinism Proof**: All 10 sources of non-determinism addressed, mitigations, test strategy (5 automated tests), continuous monitoring, failure handling [1].
9. **Greenfield Startup Runbook**: 3-week plan, day-by-day tasks, success criteria, training materials [1].
10. **Brownfield Migration Runbook**: 5-week plan, data extraction, event reconstruction, parallel run, cutover decision criteria [1].
11. **Dashboard Designs (4 Roles)**: Line Cook, Manager, Owner, Auditor dashboards with widgets and customization workflow [1].
12. **First-Order vs Second-Order Effects Analysis**: What the system does vs. how it shapes behavior, timeline (2 weeks, 2 years, 20 years) [1].
13. **Inversion Analysis (How This Fails)**: Resistance patterns, workaround scenarios,

system responses, trust preservation [1].

14. **Observability & Cost Controls**: Metric definitions, alert thresholds, CloudWatch dashboard layout, cost monitoring [1].
15. **Repair & Reconciliation Tooling**: Admin CLI commands, quarantine workflow, replay procedures, incident response runbook [1].
16. **API Specification**: REST endpoints, authentication/authorization model, idempotency requirements, rate limiting [1].
17. **Deployment Runbook**: Blue/Green deployment, rollback procedures, feature flag strategy, data migration guide [1].
18. **System Evolution Strategy**: 2 weeks, 2 years, 20 years interventions, archival plan, tech stack upgrade plan [1].
19. **Cost Model & Scaling**: Per-location cost, scaling forecast, breakeven analysis, cost optimization strategies [1].
20. **Security & Compliance**: Data encryption, access control, audit logging, GDPR considerations, health department compliance [1].

13. Critical Design Question: Graceful Failure and Trust Maintenance

When this system fails (not if), how does it fail gracefully such that trust is maintained? The core principle is: **The system never lies.** If data is uncertain (offline, pending sync, suspect calculation), it explicitly states so. Trust is maintained through transparency, not perfection [1].

Failure Scenarios and Graceful Handling

1. **Network partition between locations during dinner rush**
   - **Graceful Degradation**: Locations operate offline, queue events locally.
   - **Trust Preservation**: Events timestamped with `event_time` (when they occurred), not `processing_time` (when synced).
   - **Recovery**: Sync when reconnected, idempotency prevents duplicates [1].
2. **Bug in consumption deduction (recipe quantities doubled)**
   - **Graceful Degradation**: Variance detection triggers investigation (high variance).
   - **Trust Preservation**: Calculation version embedded in events, can replay with corrected code.

- **Recovery**: Quarantine suspect events, re-deduce consumption with fixed code, create compensating events [1].

3. **Manager quits mid-month, counts not performed for 5 days**
   - **Graceful Degradation**: System continues operating (other managers can step in).
   - **Trust Preservation**: Dashboard shows "Last count: 5 days ago" (pressure to act).
   - **Recovery**: Perform catch-up count, variance classified as "delayed_count" (separate from normal variance) [1].
4. **AWS Lambda cold start causes API timeout during sales upload**
   - **Graceful Degradation**: Retry with exponential backoff (3 attempts).
   - **Trust Preservation**: Idempotency key prevents duplicate processing.
   - **Recovery**: If all retries fail, queue to DLQ (dead-letter queue), manual replay later [1].
5. **DynamoDB write fails (transient error)**
   - **Graceful Degradation**: Event stored in local buffer, retry on next sync.
   - **Trust Preservation**: Event ID and timestamp preserved from original attempt.
   - **Recovery**: Sync when DynamoDB available, conditional write ensures no duplicates [1].

14. The Tech Stack & AWS Services (MVP Scope)

This section details the chosen tech stack and AWS services, specifically selected to maintain a $0 monthly bill for 25 users while providing enterprise-grade isolation [1].

Frontend (The User Interface)

- **Framework**: Next.js (React), written in TypeScript.
- **Styling & UI**: Tailwind CSS + Shadcn UI.
- **Hosting**: AWS Amplify (12-month Free Tier).
- **Constraint Check**: Keep bandwidth under 15 GB/month (easy for 25 users) and watch 1,000 free build minutes. Batch Git commits. Amplify automatically handles global CDN delivery; manual CloudFront configuration is not needed [1].

Backend (The Compute & Logic)

- **Framework**: Java with Spring Boot 3.
- **Hosting**: AWS Lambda via API Gateway (Always Free Tier - 1 million requests/month).

- **Cold Start Mitigation**: AWS Lambda SnapStart. GraalVM native images were explicitly rejected due to rapid consumption of free CI/CD build minutes. SnapStart caches the initialized JVM state, reducing boot times from 10 seconds to under 1 second for free [1].

Database (The Event Ledger)

- **Database**: Amazon DynamoDB (Always Free Tier - 25 GB storage, 25 Read/Write Capacity Units).
- **Design**: Single-Table Design. All locations, master ingredients, and historical events live in one table, separated by Partition Keys (PK) and Sort Keys (SK) [1].

Auth, Storage, & Utilities

- **Authentication**: Amazon Cognito (Free tier up to 50,000 monthly active users). Handles JWT generation and role-based access control (RBAC) [1].
- **File Storage**: Amazon S3 (5 GB Free Tier). Used exclusively for storing End-of-Day POS Excel sheets uploaded by users [1].
- **Cron Jobs**: Amazon EventBridge (Free tier). Triggers daily Lambdas for time-based inventory reductions (e.g., salt, sugar) [1].
- **Observability**: Amazon CloudWatch (5 GB Free Tier).
- **Crucial Setup**: Manually change the Lambda log retention policy to 7 days immediately after deployment to avoid long-term storage fees [1].
- **OCR Parsing (External)**: Google Gemini API for parsing invoice images into structured JSON data [1].

15. Functional Requirements (MVP Scope - First 90 Days)

For Phase 1 (First 90 Days), the system will exclusively handle the following business capabilities:

1. **Identity & Location Management**
    - Support multiple user roles (Owner, Store Manager, Chef, Employee) via Cognito custom claims.
    - Allow dual/multiple role assignments.
    - Support multi-location hierarchy (1 Restaurant Entity → Multiple Location Entities) [1].
2. **Master Data Management**
    - Maintain a "Master Ingredients Data" list used across all locations to streamline

onboarding.
    - Maintain "Master Menus" and "Master Recipes" that map to end-product items.
    - Support sub-recipes (e.g., a "Sauce" is an inventory item made of smaller ingredients, which is then mapped to a final menu item) [1].
3. **Inventory & Recipe Engine**
    - Track inventory per location (Ingredient, Category, Quantity, Unit of Measurement, Supplier, Average Unit Cost).
    - Map menu items to recipes with specific unit conversions (e.g., 1 sold "Fries" = minus 0.04 kg of "Potato").
    - Execute mathematical yields accurately, accounting for discrepancies between recipe units and inventory purchasing units [1].
4. **Data Ingestion & Events**
    - **Time-Based Decay**: Automatically reduce difficult-to-measure items (salt, sugar) via EventBridge daily triggers based on user-defined daily estimates [1].
    - **POS Uploads**: Accept end-of-day Excel file uploads via S3, parse sold items, and execute recipe consumption logic against inventory [1].
    - **OCR Ingestion**: Allow users to scan supplier invoices using the Gemini API to automatically draft `PurchaseReceived` events (adding to inventory) [1].

16. Non-Functional Requirements

- **Idempotency**: All write requests must contain an `idempotency_key`. The API will reject duplicate requests to ensure an ingredient is never double-deducted if a manager hits "Submit" twice [1].
- **Lazy Initialization**: Spring Boot must be configured to delay connecting to DynamoDB until the first actual user request. Connecting during SnapStart boot phase will cause a timeout error for the first user [1].
- **Secure Randomness**: Any UUIDs or security tokens generated by Spring must be hooked into SnapStart's `afterRestore` lifecycle method to ensure two concurrent users don't receive the exact same cloned session ID from the snapshot [1].
- **Degraded State**: If the Gemini OCR API fails, the system must degrade gracefully, allowing manual entry of the invoice rather than blocking the user [1].

17. API & Data Access Strategy (DynamoDB Design)

To stay within the 25 RCU/WCU limits of the free tier, database queries must be highly targeted, utilizing a Single-Table Design pattern [1].

Entity	Partition Key (PK)	Sort Key (SK)	Payload
Master Ingredient	`MASTER#ING`	`ING#[IngredientID]`	`{ name, category, default_unit }`
Location Detail	`LOC#[LocationID]`	`METADATA`	`{ owner_id, address }`
Location Inventory Balance	`LOC#[LocationID]`	`BALANCE#[IngredientID]`	`{ current_qty, avg_cost }`
Immutable Ledger Event	`LOC#[LocationID]`	`EVENT#[Timestamp]#[EventID]`	`{ event_type: "CONSUMPTION", qty_change: -2.5, causation_id: "POS_EXCEL_UPLOAD" }`

Why this works

- To load a location's current dashboard, query `PK = LOC#123` and `SK begins_with BALANCE#`.
- To audit an inventory number, query `PK = LOC#123` and `SK begins_with EVENT#` to replay the exact mathematical history [1].

18. The "Gotcha" Checklist (How to avoid failure)

- **Do not use Provisioned Concurrency**: It bypasses the Free Tier. Rely entirely on SnapStart [1].
- **Do not deploy NAT Gateways**: Keep Lambdas in standard networking setups. A NAT Gateway costs ~$32/month [1].
- **Set up Billing Alarms**: Before writing code, set an alert to email if projected spend hits $0.50 [1].
- **Enforce the 7-Day Log Rule**: Deploy new Lambda → Check CloudWatch → Set 7-day expiration [1].

By adhering to this exact architecture, the inventory system will be mathematically bulletproof, highly secure, and cost nothing to run [1].

19. Role-Based Access Control (RBAC) & Permissions Matrix

Managed via AWS Cognito Custom Claims. Users can hold dual/multiple roles, and access is strictly scoped to assigned location IDs [1].

- **Owner**: Global read access across all locations. Can define Master Ingredients/Menus. Can approve cross-location transfers and historical corrections (>30 days) [1].
- **Store Manager**: Location-scoped read/write. Can upload End-of-Day POS Excel sheets, run Gemini OCR on invoices, initiate and approve physical counts, and approve standard variance corrections (<30 days) [1].
- **Chef**: Location-scoped read/write for production. Can log recipe executions, log prep waste, and view theoretical inventory. Cannot approve variance [1].
- **Employee**: Location-scoped write-only for consumption. Can log simple usage/waste events [1].

20. Theoretical Inventory Engine (Event Ingestion)

The engine computes expected inventory at any point in time without relying on physical counts [1].

Algorithm: `theoretical_balance(ingredient, location, as_of_time) = starting_balance + Σ purchases - Σ recipe_consumptions - Σ waste + Σ transfers_in - Σ transfers_out` [1].

MVP Event Ingestion Methods

- **Manual Entry (Mobile PWA)**: Staff input counts, prep waste, or recipe executions via the stateless React frontend [1].
- **Time-Based Decay (EventBridge)**: Items like Salt and Sugar are reduced automatically. EventBridge pings a Lambda function daily at midnight, which appends a `DailyEstimatedConsumption` event based on user-defined metrics (e.g., -1 kg/day) [1].
- **End-of-Day POS Uploads (S3)**: Managers upload daily sales Excel sheets to an S3 bucket. S3 triggers a Lambda to parse the sheet, map sold menu items to Master Recipes, and append `RecipeExecuted` events to the ledger [1].
- **OCR Ingestion (Gemini API)**: Allows users to scan supplier invoices using the Gemini API to automatically draft `PurchaseReceived` events (adding to inventory) [1].

References

[1] Pasted Content: Comprehensive Enterprise-Grade Multi-Location Inventory Control System Design. (n.d.). Provided by user as pasted_content.txt.